

Jenkins to GitLab CI/CD Migration

Maven Application | Practical Runbook

Prepared from the completed Windows-based learning lab | Reference for repeat use

1. Purpose and scope

This runbook records the practical migration of a small Maven application from a Jenkins pipeline to GitLab CI/CD. It documents the source project, legacy Jenkins configuration, GitLab repository, Windows GitLab Runner setup, CI configuration, troubleshooting, validation, and optional GitHub remote.

Completed scope: CI build, unit test, package, JUnit report collection, and GitLab job-artifact upload. Publishing to Artifactory, containerization, and deployment are future extensions—not completed parts of this lab.

2. Environment used

Item	Lab value
Operating system	Windows 11
Terminal	Git Bash (UCRT64) and Administrator PowerShell
Java	Eclipse Temurin JDK 21
Maven	Apache Maven 3.9.16
Jenkins	Jenkins running directly on Windows
GitLab	GitLab.com project
GitLab Runner	19.4.1, Windows service
Runner executor / shell	shell / powershell
Runner tag	windows
Local project	F:\migrating_jenkins_gitlab_repos\first-project
GitLab remote	https://gitlab.com/muthyal-group/first-project.git
Branch	main
Runner install folder	C:\GitLab-Runner

Adapt paths, versions, project URLs, runner tags, and branch names for other environments. Never place access tokens, passwords, private keys, OTPs, or payment details in this document or source control.

3. Migration architecture

Before:

```
Developer → GitLab repository → Jenkins (Jenkinsfile)
├─ Build
├─ Test
└─ Package → Jenkins archived JAR
```

After:

```
Developer → GitLab repository → .gitlab-ci.yml → GitLab Runner
├─ Build
├─ Test + JUnit report
└─ Package → GitLab job artifact
```

The same Maven project and commands were retained; the pipeline orchestration and artifact handling moved from Jenkins to GitLab CI/CD.

4. Maven project layout

```
first-project/
├─ README.md
```

```

├─ pom.xml
├─ Jenkinsfile
├─ .gitlab-ci.yml
├─ src/
│   └─ main/java/com/example/App.java
└─ test/java/com/example/AppTest.java

```

The application exposes `App.greet()`, which returns “Hello from Maven CI/CD!”. A JUnit 5 test checks this value. The Maven coordinates are `com.example:first-project:1.0.0`. The POM uses Java release 17, while the installed runtime/toolchain is JDK 21.

5. Maven configuration

The project POM used these key settings and dependencies:

```

<groupId>com.example</groupId>
<artifactId>first-project</artifactId>
<version>1.0.0</version>

<properties>
  <maven.compiler.release>17</maven.compiler.release>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.13.0</version>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
    </plugin>
  </plugins>
</build>

```

The complete `pom.xml` should be retained in the repository. The excerpt above records the project’s key configuration, not a replacement for the full file.

6. Validate the project locally

In Git Bash, move to the project and confirm Java and Maven are available:

```

cd /f/migrating_jenkins_gitlab_repos/first-project
java -version
mvn -version
mvn clean compile

```

```
mvn test
mvn package -DskipTests
```

Expected package output: target/first-project-1.0.0.jar. Resolve local build failures before moving to CI configuration.

7. Create and connect the GitLab repository

1. Create the GitLab project first-project under the intended namespace. For a new repository, avoid initializing extra files if the local repository already contains a README or other initial commit.
2. From the project directory, inspect the existing Git configuration before adding remotes.
3. Ensure the main branch is used and push the project to GitLab.

```
git status
git branch --show-current
git remote -v

# If this is a newly initialized local repository:
git init
git branch -M main
git remote add origin https://gitlab.com/muthyal-group/first-project.git
git add .
git commit -m "Initial Maven project"
git push -u origin main
```

Do not repeat git init, add an existing remote again, or create a duplicate initial commit if the repository is already connected. In this lab, the local project was successfully pushed to the GitLab URL above. For a private repository, authenticate securely using Git Credential Manager or an appropriately scoped access token; never embed a token in the remote URL.

8. Configure and validate the legacy Jenkins pipeline

Jenkins was configured with tool installations named JDK21 and Maven3. The Jenkins job used Pipeline script from SCM, Git as SCM, the GitLab repository URL, credentials stored in Jenkins, branch */main, and script path Jenkinsfile.

The Jenkinsfile used in the lab:

```
pipeline {
    agent any

    tools {
        jdk 'JDK21'
        maven 'Maven3'
    }

    stages {
        stage('Build') {
            steps {
                bat 'mvn clean compile'
            }
        }

        stage('Test') {
            steps {
                bat 'mvn test'
            }
        }

        stage('Package') {
            steps {
```

```

        bat 'mvn package -DskipTests'
    }
}
}

post {
    always {
        junit 'target/surefire-reports/*.xml'
        archiveArtifacts artifacts: 'target/*.jar',
            fingerprint: true
    }
}
}

```

Run the Jenkins job and confirm the Build, Test, and Package stages succeed, JUnit results are recorded, and target/first-project-1.0.0.jar is archived. The lab's Jenkins run completed successfully.

9. Map Jenkins concepts to GitLab CI/CD

Jenkins	GitLab CI/CD equivalent used
Jenkinsfile	.gitlab-ci.yml
Jenkins stage	GitLab stage; individual execution is a job
Jenkins agent	GitLab Runner
bat command	script command executed by configured runner shell
archiveArtifacts	artifacts
Jenkins JUnit publisher	artifacts:reports:junit
Jenkins credentials	GitLab CI/CD variables or a suitable secret integration
Jenkins workspace	Runner job working directory

GitLab pipelines are defined using stages and jobs. Jobs in later stages run after earlier stages succeed by default; jobs in the same stage may run in parallel.

10. Add the GitLab CI configuration

Create .gitlab-ci.yml in the repository root. The lab configuration was:

```

stages:
  - build
  - test
  - package

build_job:
  stage: build
  tags:
    - windows
  script:
    - mvn clean compile

test_job:
  stage: test
  tags:
    - windows
  script:
    - mvn test
  artifacts:
    when: always
    reports:
      junit:
        - target/surefire-reports/TEST-*.xml

```

```
package_job:
  stage: package
  tags:
    - windows
  script:
    - mvn package -DskipTests
  artifacts:
    paths:
      - target/*.jar
    expire_in: 1 week
```

The windows tag must match a runner tag. The JUnit report path matches Maven Surefire's XML report location. The package job stores the generated JAR as a GitLab job artifact with a one-week expiration. Artifacts are pipeline outputs for download/transfer; they are not a permanent artifact repository such as Artifactory.

11. Commit the GitLab pipeline file

```
git status
git add .gitlab-ci.yml
git commit -m "Migrate Jenkins pipeline to GitLab CI"
git push origin main
```

If Git reports nothing to commit, inspect git status and confirm the file was already committed. The lab's .gitlab-ci.yml was pushed to main.

12. Install and register the Windows GitLab Runner

The project used a self-managed Windows Shell Runner because Docker Desktop was not installed on the host. The runner was installed under C:\GitLab-Runner and registered to the GitLab project using GitLab's runner creation workflow. The registration flow provided a runner authentication token (commonly prefixed glrt-); treat it as a secret.

4. Create a project runner in GitLab under Settings → CI/CD → Runners and assign the tag windows. Use the current GitLab UI instructions for runner registration; do not rely on deprecated registration-token instructions.
5. Install the runner binary in C:\GitLab-Runner and register it with the project using the provided URL/token and the shell executor.
6. Install and start it as a Windows service from Administrator PowerShell, if it was not installed as a service during registration.
7. Confirm the runner appears online and its tag matches the jobs' windows tag.

```
cd C:\GitLab-Runner
.\gitlab-runner.exe install
.\gitlab-runner.exe start
.\gitlab-runner.exe status
```

In the completed lab, GitLab showed the project runner as Online/Idle. Do not publish config.toml or reveal its token.

13. Configure the runner shell

The first pipeline attempt failed with:

```
exec: "pwsh": executable file not found in %PATH%
```

Cause: the runner selected PowerShell Core (pwsh), which was not available on the host. The fix was to edit C:\GitLab-Runner\config.toml and configure the Shell executor to use Windows PowerShell:

```
executor = "shell"
shell = "powershell"
```

After saving the file, restart the runner service:

```
cd C:\GitLab-Runner
.\gitlab-runner.exe restart
.\gitlab-runner.exe status
```

The corrected job log confirmed: "Using Shell (powershell) executor...".

14. Ensure Maven is available to the runner service

The next pipeline attempt failed because `mvn` was not recognized. The interactive user environment had Maven, but the Windows service environment did not have Maven's bin directory on its PATH.

The Maven installation used in this lab:

```
Maven home:
C:\Users\HP\AppData\Local\Programs\Apache\apache-maven-3.9.16

Maven bin:
C:\Users\HP\AppData\Local\Programs\Apache\apache-maven-3.9.16\bin
```

Check the Maven path from PowerShell:

```
where.exe mvn
```

If the service cannot resolve Maven, add the actual Maven bin path to the machine-level PATH and set MAVEN_HOME from Administrator PowerShell. Example for this lab (adjust if installed elsewhere):

```
[Environment]::SetEnvironmentVariable(
    "MAVEN_HOME",
    "C:\Users\HP\AppData\Local\Programs\Apache\apache-maven-3.9.16",
    "Machine"
)

$machinePath = [Environment]::GetEnvironmentVariable("Path", "Machine")
$mavenBin = "C:\Users\HP\AppData\Local\Programs\Apache\apache-maven-3.9.16\bin"

if (($machinePath -split ';') -notcontains $mavenBin) {
    [Environment]::SetEnvironmentVariable(
        "Path",
        "$machinePath;$mavenBin",
        "Machine"
    )
}
```

Restart the GitLab Runner service after changing machine environment variables so the service receives the updated environment. Verify Maven and Java are accessible to the runner account. Avoid depending on a user-profile-only installation if the runner service runs under another account.

15. GitLab account verification encountered

GitLab.com displayed an account identity-verification requirement before allowing pipeline execution. This is separate from project YAML and runner setup. Follow the verification options presented by GitLab and review the current verification/billing screens carefully. Completing identity verification alone should not be confused with purchasing a paid plan. Do not enter payment details unless comfortable with the provider's stated process.

16. Run the pipeline and verify each job

8. Open the GitLab project and navigate to Build → Pipelines (menu labels can change slightly).
9. Run or inspect the pipeline for the main branch after .gitlab-ci.yml is committed.
10. Open each job log and verify the expected Maven command and successful completion.

11. Confirm the test job publishes JUnit XML results from target/surefire-reports/.
12. Open the package job's artifacts and verify target/first-project-1.0.0.jar is available.

Job	Command	Expected output
build_job	mvn clean compile	Compiled application; job succeeds
test_job	mvn test	Tests pass; JUnit XML report is collected
package_job	mvn package -DskipTests	JAR created and uploaded as job artifact

Successful lab evidence in the package log included `target/\*.jar: found 1 matching artifact files and directories`, artifact upload returning `201 Created`, and `Job succeeded`. All three jobs—Build, Test, and Package—completed successfully.

17. Final working pipeline

```
GitLab repository (main)
|
v
.gitlab-ci.yml
|
v
Windows GitLab Runner (shell / powershell, tag: windows)
|
+--> build_job: mvn clean compile
|
+--> test_job: mvn test
|               + JUnit report
|
+--> package_job: mvn package -DskipTests
|
|               v
|               target/*.jar
|               |
|               v
|               GitLab artifact
```

Result: the Jenkins Maven CI flow was reproduced and validated in GitLab CI/CD. The Jenkinsfile was retained in the repository as a legacy/reference pipeline file; GitLab uses .gitlab-ci.yml.

18. Optional GitHub remote

The local repository was also pushed to GitHub as a second remote. This is optional and does not replace GitLab or change the GitLab pipeline.

```
git remote -v
# Example remotes:
# origin https://gitlab.com/muthyal-group/first-project.git
# github https://github.com/KASEVADDI/Jenkins-Gitlab-migration.git

# Keep GitLab as the default upstream:
git branch --set-upstream-to=origin/main main

# Push to GitLab:
git push

# Push explicitly to GitHub:
git push github main
```

The lab encountered a non-fast-forward rejection because the GitHub remote already contained a commit. Fetch and inspect remote history first. Only if the remote contains no work that needs preserving may a deliberate force push be considered; do not force-push a shared repository.

19. Troubleshooting record

Symptom	Likely cause	Resolution used or check
exec: "pwsh": executable file not found	Runner selected pwsh, but it was unavailable.	Set shell = "powershell" in config.toml; restart runner.
mvn is not recognized	Maven not available in the runner service PATH.	Add correct Maven bin directory to machine PATH; restart service; verify tool access.
Job is stuck / runner not picked	Runner offline, paused, or tags do not match.	Check runner status and ensure runner/job tag both include windows.
Git push rejected: fetch first	Remote branch has commits absent locally.	Fetch and inspect; integrate changes or carefully replace only disposable remote history.
git status tracks github/main	Local branch upstream points to GitHub.	Run git branch --set-upstream-to=origin/main main.
GitLab requires account verification	GitLab.com account-level gate.	Follow GitLab's displayed verification process; this is not a YAML syntax error.

20. Security and operational cautions

- Store GitLab credentials in Jenkins Credentials and GitLab secrets in CI/CD variables or a suitable secret manager—not in repository files.
- Never commit runner config.toml if it contains authentication material.
- Do not paste tokens, passwords, private keys, CVVs, or OTPs into tickets, logs, screenshots, or chat.
- Use least-privilege access tokens and rotate any secret that is exposed.
- A Shell Runner executes jobs directly on its host. Use a dedicated, controlled runner machine and restrict which projects/users can run jobs on it.
- GitLab job artifacts have a retention period; the lab configuration sets one week. Download or publish outputs elsewhere if longer retention is required.
- Review pipeline permissions, protected branches, and deployment approvals before adapting this learning setup for production.

21. Future extension plan

The following are planned enhancements and were not part of the completed CI migration:

13. Publish the Maven JAR to JFrog Artifactory, using securely stored repository credentials.
14. Add code-quality and security scanning such as SonarQube and software-composition analysis where relevant.
15. Build and scan a Docker image; publish it to an approved container registry.
16. Add deployment jobs for Dev, QA/UAT, and Production with environment protections and approvals.
17. Introduce Terraform and/or Ansible for infrastructure/configuration automation if required by the target architecture.
18. Integrate a secret manager and notifications, and add clear failure handling and rollback procedures.

Implement and validate each extension independently; do not treat planned work as completed experience.

22. Interview summary

I built a Maven application and validated its Jenkins pipeline with Build, Test and Package stages. I then migrated the CI orchestration to GitLab CI/CD by defining equivalent jobs in .gitlab-ci.yml.

I configured a Windows GitLab Shell Runner, corrected its shell configuration to use Windows PowerShell, and made Java and Maven available to the runner service. I added JUnit report collection and configured the packaged JAR as a GitLab job artifact. The Build, Test and Package jobs all succeeded, and the JAR upload was confirmed. This was a hands-on learning lab, not a production migration.

Describe the work as a practical lab unless it was also performed in a real production environment.

23. Reuse checklist

- ☐ Confirm project builds locally with the intended JDK and Maven.
- ☐ Record the existing Jenkins stages, tools, triggers, credentials, reports, and artifacts.
- ☐ Confirm GitLab repository and branch.
- ☐ Preserve the Jenkinsfile as reference if required.
- ☐ Translate pipeline stages/jobs and commands into .gitlab-ci.yml.
- ☐ Configure runner executor, shell, tags, service account, and tool PATH.
- ☐ Store secrets securely and verify permissions.
- ☐ Run pipeline and inspect every job log.
- ☐ Verify tests, JUnit report, packaged artifact, and retention.
- ☐ Document known gaps, rollback approach, and next extensions.

24. Version and accuracy note

This runbook reflects the configuration and outcomes described for the completed lab. GitLab's UI and runner-registration workflow can evolve; use the current instructions displayed by your GitLab instance when repeating setup. Confirm exact paths and tool versions on the target host before executing commands.